

CIS 9590 — TERM PROJECT FINAL REPORT

MediSync

Medical Appointment & Patient Scheduling System

A Centralized, Web-Based Scheduling Platform for Outpatient Clinics

Prepared by Group 3

Gil • Maggie • Mehdi • Raylene

Course: CIS 9590 — Systems Analysis & Design

Submission: Final Report & Working Prototype Demonstration

Live Demo: <https://maps-system-three.vercel.app/login>

July 2026

Table of Contents

Executive Summary	4
Phase 1: Problem Definition	5
1.1 Project Summary	5
1.2 The System Setting	5
1.3 Describe the Project Environment	5
Systems History	5
System Users and Functional Responsibilities	5
1.4 The Systems Description	6
Principal Inputs and Their Forms	6
Data Handling Process	6
Outputs	6
1.5 Goals for the System	6
1.6 Data Gathering Plan	6
Phase 2: System Representation (Logical Design)	8
2.1 Design Summary	8
2.2 Context-Level Diagrams	8
Level 0 — System Overview	8
Level 1 — Major Processes	9
Level 2 — Decomposition of Process 2.0 (Manage Appointments)	9
2.3 Menu Hierarchy	10
Patient Menu	10
Doctor Menu	10
Administrator Menu	10
2.4 Report Design	11
Sample Report Layout — Daily Appointment Report	11
Sample Report Layout — Doctor Workload Report	11
Phase 3: Project & Systems Analysis	13
3.1 Project Cost Analysis	13
3.2 Labor and Resources	13
Labor	13
Resources — Hardware	13
Resources — Software	13
3.3 Project Timeline	14
3.4 Use Case Diagram	14
Use Case Narratives	14
3.5 Major Processes and Decision Logic	15
3.6 Database Design / Schema	15
Table Summary	16
3.7 Sample SQL Queries	16
Create a new patient account	17
Search available doctors by specialty	17
Book an appointment (with conflict check)	17
Cancel an appointment	17

Daily Appointment Report query	17	
Doctor Workload Report query	17	
Phase 4: Build Your System / Prototype		19
4.1 Demo Summary	19	
4.2 Features Demonstrated	19	
Navigation	19	
User Input (New Transaction)	19	
Edit / Change Transaction	19	
Query the Data (Reports)	19	
4.3 Demo Story	19	
4.4 Implemented Features (Prototype Scope)	20	
4.5 Application Screenshots	20	
Login	20	
Patient View	21	
Doctor View	24	
Administrator View	26	
Phase 5: System Implementation		29
5.1 Executive Summary — Lessons Learned	29	
5.2 Hardware / Software	29	
Platform Decisions	29	
5.3 Testing / Training	30	
Test Plan	30	
Training Plan	30	
5.4 Implementation Plan	30	
5.5 Post-Launch / Support Plan	30	
5.6 Future Modifications / Features	30	
Appendix		32
Appendix A — Gantt Chart	32	
Appendix B — Project Charter	32	
Appendix C — Meeting Logs	33	
Appendix D — Change Log	33	
Appendix E — Issues / Risk Log	34	
Appendix F — Variance Report	34	

Executive Summary

Outpatient medical practices continue to rely heavily on phone calls, paper logs, and disconnected spreadsheets to manage patient scheduling. This manual approach produces double-booked time slots, missed appointments, inconsistent record-keeping, and little to no visibility into clinic performance. MediSync is a web-based Medical Appointment and Patient Scheduling System designed to replace these manual workflows with a single, centralized platform that connects patients, doctors, and clinic administrators.

The system allows patients to create secure accounts, search for doctors by specialty, book and manage their own appointments, and review visit history, all without picking up a phone. Doctors gain a clear view of their daily schedules and can record patient notes directly in the system. Administrators are given full oversight of physician schedules, appointment approvals, and operational reporting, enabling data-driven staffing and performance decisions.

This report documents the full systems analysis and design lifecycle for MediSync: the business problem and system setting, the logical system design (context diagrams, menu hierarchy, and report design), the project and systems analysis (cost estimate, use cases, and database schema), the working prototype and its core demonstrated features, and the lessons learned during implementation. A working prototype has been deployed and is available for live demonstration at <https://maps-system-three.vercel.app/login>.

Team: Gil, Maggie, Mehdi, and Raylene (Group 3)

Technology Stack: React, TypeScript, and Tailwind CSS on the front end; Node.js and Express on the back end; PostgreSQL (hosted on Neon) for data persistence; JWT and bcrypt for authentication and security; Git/GitHub for version control; and Vercel for deployment.

Phase 1: Problem Definition

1.1 Project Summary

MediSync is a web-based Medical Appointment and Patient Scheduling System built for outpatient clinics and small-to-medium medical practices. The system replaces manual, phone-based scheduling with a self-service digital platform where patients register for an account, search for doctors by specialty, book or cancel appointments, and view their appointment history. Doctors use the system to view their daily schedules and add patient visit notes. Clinic administrators manage physician profiles and schedules, approve or cancel appointment requests, and generate operational reports covering appointment volume, cancellations, and provider utilization. By centralizing these workflows in a single application, MediSync reduces scheduling conflicts, cuts down on administrative overhead, and gives clinic managers real-time visibility into how the practice is performing.

1.2 The System Setting

Industry: Healthcare — outpatient clinics and ambulatory medical practices.

MediSync is designed for small and medium-sized medical offices (for example, a multi-provider family practice or a specialty clinic) that currently lack an integrated scheduling and patient-management system. These practices are too small to justify the cost of a full enterprise Electronic Health Record (EHR) suite, but still need a reliable way to manage appointments, provider schedules, and basic patient records digitally.

1.3 Describe the Project Environment

Systems History

MediSync is a new system. There is no prior digital scheduling platform in place at the target clinics — the current process is a manual, phone- and spreadsheet-based workflow. Because there is no existing system to migrate from, MediSync is designed as a greenfield prototype: it demonstrates the core scheduling workflow end-to-end without needing to preserve legacy data or interfaces. The impact of introducing MediSync is expected to be significant: clinics moving from phone/spreadsheet scheduling to a centralized system typically see fewer double-bookings, faster patient intake, and dramatically improved reporting visibility, since none of that data was previously captured in a structured, queryable form.

System Users and Functional Responsibilities

- Patients (public users) — register an account, search for providers, book/modify/cancel their own appointments, and view their appointment history.
- Doctors (clinical staff) — view their daily/weekly schedules, review patient information for upcoming visits, and record clinical notes.
- Clinic Administrators (business unit / operations staff) — manage doctor profiles and schedules, approve or cancel appointment requests, and run operational reports for clinic management.

1.4 The Systems Description

Principal Inputs and Their Forms

- Electronic: patient registration data, login credentials, appointment requests, doctor availability updates, and administrator changes — all entered through web forms.
- System-generated: appointment status changes, timestamps, and confirmation records created automatically as users interact with the platform.

Data Handling Process

The React/TypeScript front end collects and validates user input (registration forms, appointment search filters, booking forms) and passes it to a Node.js/Express REST API. The API applies business logic — for example, checking that a requested appointment slot does not conflict with an existing booking — before reading from or writing to the PostgreSQL database hosted on Neon. Authentication is handled with JSON Web Tokens (JWT) and bcrypt password hashing, and role-based access control (RBAC) ensures patients, doctors, and administrators only see the screens and data appropriate to their role.

Outputs

- Database updates: new/modified patient profiles, appointments, doctor schedules, and prescription refill requests.
- On-screen confirmations: appointment confirmations, booking conflicts, and status changes shown to the user in real time.
- Reports: daily appointment reports, doctor workload reports, patient visit history, and cancellation reports, viewable on screen and exportable by administrators.

1.5 Goals for the System

Goal 1 — Reduce scheduling conflicts through centralized booking. Because every appointment is written to a single shared database with server-side conflict checking, MediSync prevents the double-booking that occurs when multiple staff members independently manage the same provider's calendar by phone or spreadsheet.

Goal 2 — Improve the patient experience with 24/7 self-service scheduling. Patients are no longer limited to calling during office hours. They can search for a provider, see real-time availability, and book an appointment at any time, which is expected to reduce time-to-appointment and improve patient satisfaction.

Goal 3 — Give clinic managers real-time, data-driven operational reports. Because every appointment and cancellation is captured electronically, administrators can generate reports on appointment volume, no-show/cancellation rates, and provider utilization on demand, supporting better staffing and scheduling decisions than the manual process ever allowed.

1.6 Data Gathering Plan

The team's requirements were developed through a combination of course-provided guidelines, structured team brainstorming sessions, and research into how comparable scheduling systems in healthcare and other reservation-based industries are structured. Specific activities included:

- Requirements workshops — the team met to translate the CIS 9590 project guidelines into a concrete set of patient, doctor, and administrator functions.
- Process walkthroughs — the team mapped out, step by step, how a patient currently books an appointment by phone versus how that same interaction should work in MediSync, in order to identify the data each step requires.
- Comparative research — the team reviewed publicly available information on outpatient scheduling and patient-portal workflows (like Zocdoc) to validate that the proposed feature set (search, booking, cancellation, history, reporting) reflects real-world clinic needs.
- Iterative feedback — the initial team presentation (Systems Vision Document) was used to validate scope with the instructor before development began, and the feature list was refined based on that feedback.

Phase 2: System Representation (Logical Design)

2.1 Design Summary

MediSync's prototype implements the core scheduling workflow end-to-end: account registration and role-based login, doctor search by specialty, appointment booking with conflict prevention, appointment editing/cancellation, and an administrator dashboard with operational reporting. The logical design below shows how data flows through the system (Context Diagrams, Levels 0-2), how users navigate the application (Menu Hierarchy), and what output the system produces (Report Design).

Beyond the prototype, a complete production version of MediSync would add automated SMS/email reminders, an analytics dashboard with trend visualizations, insurance verification, and a patient-facing mobile app — all of which are documented as future enhancements in Phase 5 and were intentionally scoped out of the prototype to keep the initial build focused on proving the core scheduling workflow.

2.2 Context-Level Diagrams

Level 0 — System Overview

The Level 0 diagram treats MediSync as a single process and shows its boundary: the four external entities (Patient, Doctor, Clinic Administrator, and the Email/SMS Reminder Service) and the high-level data flows between each entity and the system.

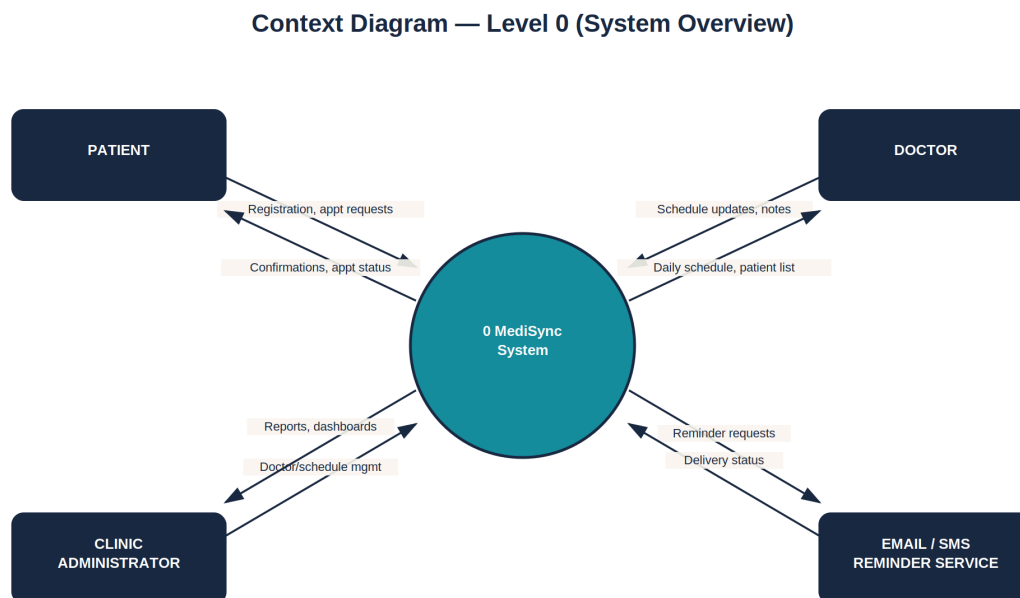


Figure 1. Level 0 Context Diagram — System Overview

Level 1 — Major Processes

The Level 1 diagram decomposes the single MediSync process into its four major functional processes — Manage Patient Accounts, Manage Appointments, Manage Doctor Schedules, and Generate Reports — and introduces the system's core data stores: Patients, Doctors/Specialties, Appointments, and Reports.

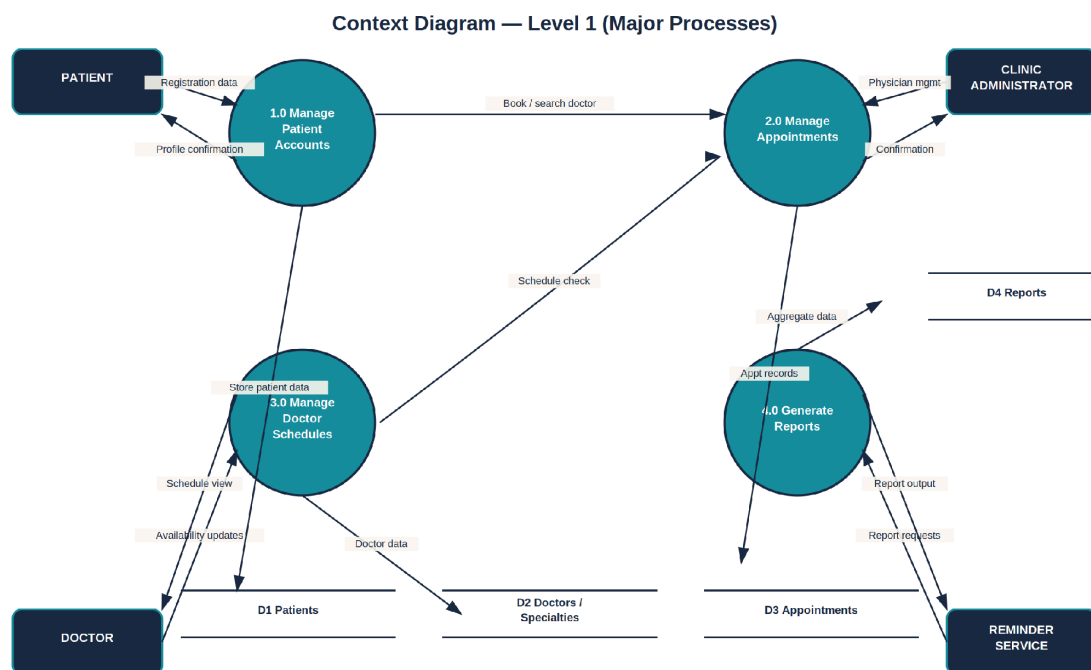


Figure 2. Level 1 Context Diagram — Major Processes

Level 2 — Decomposition of Process 2.0 (Manage Appointments)

The Level 2 diagram drills into the Manage Appointments process, which is the most data- and logic-intensive part of the system. It is broken into four sub-processes: Search Doctor Availability, Book Appointment, Modify/Cancel Appointment, and Send Confirmation, each of which reads from or writes to the Doctors/Specialties and Appointments data stores.

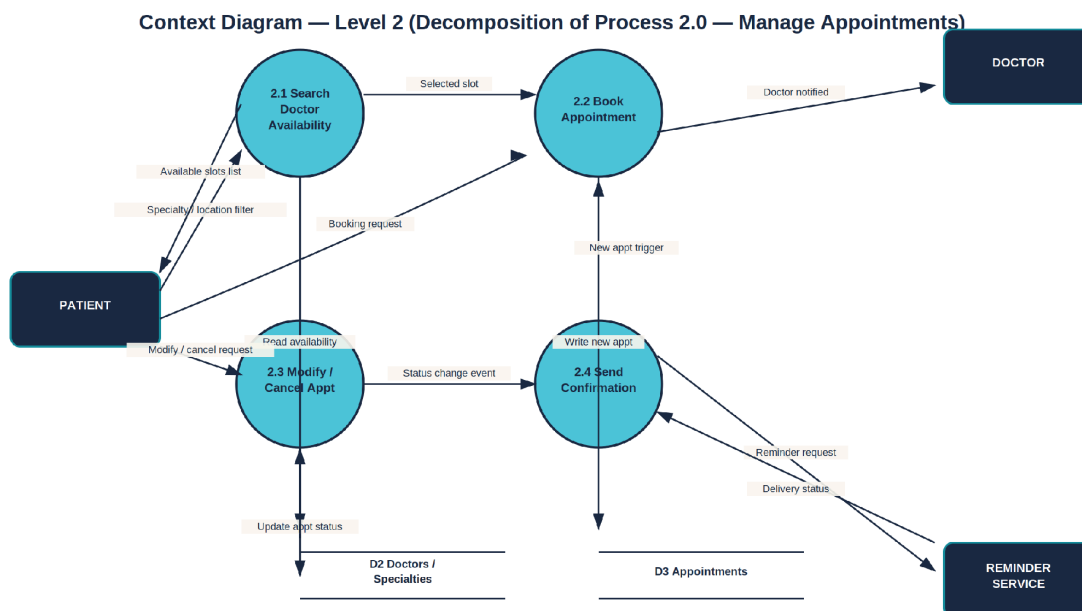


Figure 3. Level 2 Context Diagram — Manage Appointments Decomposition

2.3 Menu Hierarchy

MediSync presents three role-specific navigation menus, shown to a user only after successful login and role verification:

Patient Menu

- Login / Register
- Dashboard
- Find a Doctor → Search by Specialty, Search by Location
- Book Appointment → Select Doctor, Select Date/Time, Confirm
- My Appointments → Upcoming, Past / History, Modify, Cancel
- Prescription Refills → Request Refill, Refill Status
- My Profile → Personal Info, Insurance Info, Password

Doctor Menu

- Dashboard
- My Schedule → Today, Weekly View
- My Patients → Patient List, Visit Notes
- Prescription Requests → Approve / Deny

Administrator Menu

- Dashboard

- Doctors → Add Doctor, Edit Doctor, Manage Schedules
- Patients → View Patient Records
- Appointments → Approve / Cancel, All Appointments
- Reports → Daily Appointments, Doctor Workload, Cancellations, Patient Visit History, Provider Utilization

2.4 Report Design

MediSync generates the following operational reports. All reports are available to Clinic Administrators from the Reports menu; the Daily Appointment Report and Doctor Workload Report are also available to individual doctors for their own schedule.

Report Name	Data Produced	Who Can Run It	Purpose
Daily Appointment Report	All appointments for a selected date, by doctor and time slot	Admin, Doctor	Front-desk and provider view of the day's schedule
Doctor Workload Report	Appointment count and hours booked per doctor over a date range	Admin, Doctor	Balance patient load across providers
Patient Visit History Report	Chronological list of a patient's past visits and notes	Admin, Doctor	Clinical continuity and patient record review
Appointment Cancellation Report	Cancelled/no-show appointments over a date range, with reasons	Admin	Identify no-show trends and reduce lost capacity
Provider Utilization Report	% of available appointment slots booked, per doctor	Admin	Staffing and scheduling decisions

Sample Report Layout — Daily Appointment Report

Below is a representative layout of the Daily Appointment Report as it would be generated from the database for a single clinic day.

Time	Patient	Doctor	Specialty	Status
9:00 AM	J. Alvarez	Dr. S. Kim	Family Medicine	Confirmed
9:30 AM	M. Chen	Dr. R. Osei	Cardiology	Confirmed
10:00 AM	L. Torres	Dr. S. Kim	Family Medicine	Pending
10:30 AM	A. Patel	Dr. J. Diaz	Dermatology	Cancelled
11:00 AM	K. Nguyen	Dr. R. Osei	Cardiology	Confirmed

Sample Report Layout — Doctor Workload Report

Doctor	Specialty	Appointments (This Week)	Booked Hours	Utilization %
Dr. S. Kim	Family Medicine	34	28.5	89%
Dr. R. Osei	Cardiology	27	22.0	78%
Dr. J. Diaz	Dermatology	19	15.0	63%

Phase 3: Project & Systems Analysis

3.1 Project Cost Analysis

The following is a rough-order-of-magnitude cost estimate for taking MediSync from prototype to a production-ready, HIPAA-compliant system, based on comparable small-scale healthcare software projects.

Cost Category	Estimated Range	Notes
Development (one-time)	\$30,000 – \$50,000	Front-end/back-end build, QA, project management
HIPAA Compliance & Security	\$10,000 – \$20,000	Security audit, encryption, access controls, compliance review
Annual Maintenance & Operations	\$20,000 – \$30,000+	Hosting, database, support, bug fixes, feature updates
Total Year 1	\$60,000 – \$100,000	Development + compliance + first year of operations

Cost Feasibility: For a clinic currently losing revenue to no-shows and double-bookings, or paying staff for hours of manual phone scheduling, the Year 1 investment is recoverable through reduced administrative labor, higher appointment throughput, and lower no-show rates driven by automated reminders. Because the prototype is built on low-cost, managed cloud services (Neon PostgreSQL, Vercel), ongoing infrastructure costs stay low relative to a self-hosted enterprise EHR.

3.2 Labor and Resources

Labor

- 2 Front-end developers (React/TypeScript/Tailwind UI)
- 2 Back-end developers (Node.js/Express API, PostgreSQL schema and queries)
- Shared responsibilities across the 4-person team for QA/testing, project management, and documentation

Resources — Hardware

- Individual development laptops (team-owned)
- No dedicated on-premise servers required — the prototype uses cloud-hosted infrastructure

Resources — Software

- Visual Studio Code (IDE)
- Git / GitHub (version control and collaboration)
- Neon (managed PostgreSQL database)
- Vercel (application hosting and deployment)

- Node.js, npm/React ecosystem packages

3.3 Project Timeline

The prototype was planned and built over a 12-week development cycle, organized into three phases (see the Gantt chart in Appendix A for the full task-level timeline):

- Phase 1 — Foundation (Weeks 1-4): database design, authentication, admin panel framework, doctor profile setup.
- Phase 2 — Core Features (Weeks 5-8): doctor search, appointment booking, calendar integration, appointment history.
- Phase 3 — Enhancement (Weeks 9-12): automated reminders, reporting/analytics dashboard, user testing, deployment.

3.4 Use Case Diagram

The use case diagram below shows the three actors — Patient, Doctor, and Administrator — and their interactions with the MediSync system boundary.

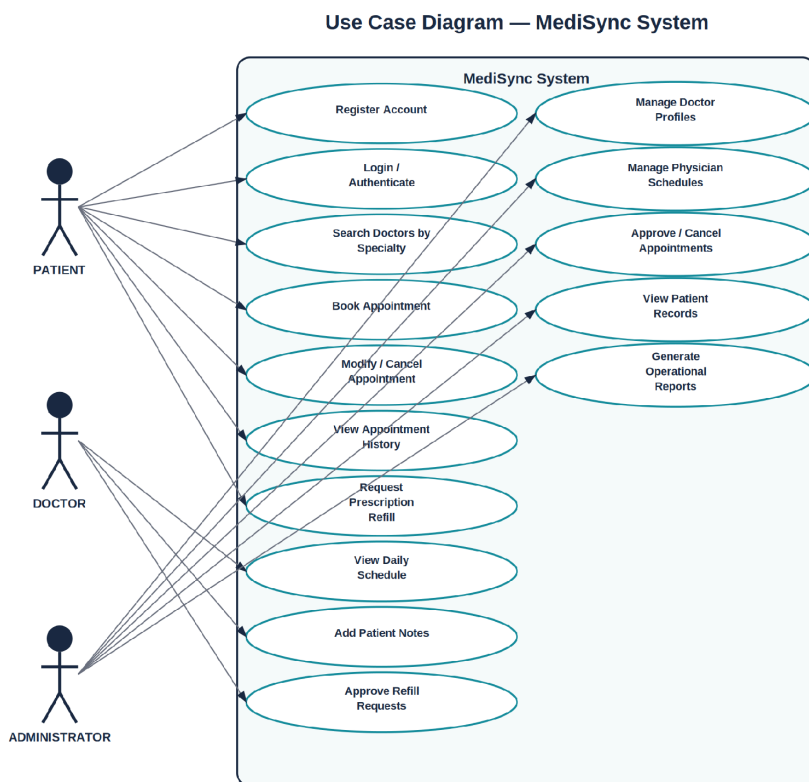


Figure 4. Use Case Diagram — MediSync System

Use Case Narratives

Use Case 1 — Patient Books an Appointment. A registered patient logs into MediSync, searches for a doctor by specialty (e.g., Cardiology), reviews real-time availability, and selects an open time slot. The system validates that the slot is still open, writes the new appointment to the Appointments table with a "Confirmed" or "Pending" status, and displays a confirmation to the patient. This corresponds to sub-processes 2.1 and 2.2 in the Level 2 context diagram.

Use Case 2 — Administrator Manages a Physician's Schedule. A clinic administrator logs in, navigates to the Doctors section, and updates a physician's available hours (for example, blocking out a holiday or adding an extra clinic day). The system updates the Doctors/Specialties data, which immediately affects what time slots patients see when they search for that doctor — directly linking process 3.0 (Manage Doctor Schedules) to process 2.0 (Manage Appointments).

Use Case 3 — Doctor Reviews Schedule and Adds Patient Notes. A doctor logs in and views their daily schedule, which lists each scheduled patient and appointment reason. After a visit, the doctor adds a clinical note associated with that appointment, which is stored as part of the visit record for future reference by that provider or an administrator.

Use Case 4 — Administrator Generates an Operational Report. An administrator selects a report type (for example, the Appointment Cancellation Report) and a date range from the Reports menu. The system queries the Appointments table, aggregates the results, and renders them as a formatted report on screen, supporting staffing and scheduling decisions (process 4.0).

3.5 Major Processes and Decision Logic

MediSync's core business logic follows simple conditional rules layered on top of the four major processes shown in the Level 1 diagram:

- If a patient requests a time slot that is already booked → the system rejects the request and returns the next available slots (prevents double-booking).
- If a patient cancels an appointment → the system marks the appointment "Cancelled," frees the time slot, and logs the cancellation for the Cancellation Report.
- If an administrator blocks a doctor's availability (e.g., vacation) → any pending requests for that window are flagged for the patient to reschedule.
- If a doctor approves a prescription refill request → the system updates the prescription status and notifies the patient; if denied, the patient is prompted to schedule a follow-up visit.

3.6 Database Design / Schema

MediSync's PostgreSQL database is organized around seven core tables: Patients, Doctors, Specialties, Appointments, Prescriptions, Visit_Records, and Admins. The Entity Relationship Diagram (ERD) below shows each table's key fields and the relationships between them.

Entity Relationship Diagram (ERD) — MediSync Database

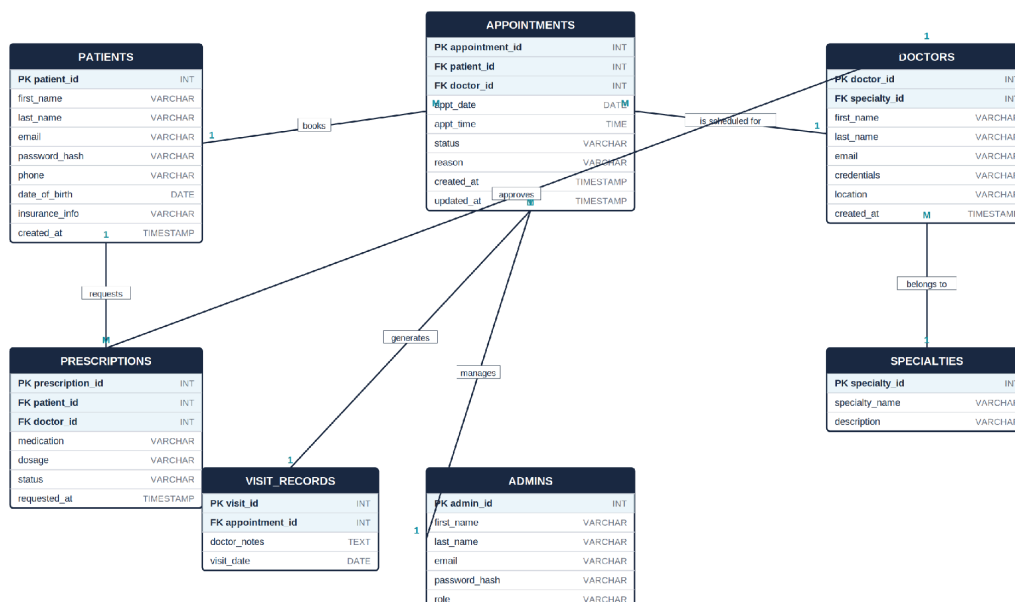


Figure 5. Entity Relationship Diagram (ERD) — MediSync Database

Table Summary

Table	Purpose	Key Relationships
Patients	Stores patient account and demographic data	1 patient → many Appointments, many Prescriptions
Doctors	Stores physician profile and credential data	many Doctors → 1 Specialty; 1 doctor → many Appointments
Specialties	Lookup table of medical specialties	1 Specialty → many Doctors
Appointments	Core transactional table linking patients and doctors to a date/time	FK to Patients, FK to Doctors; 1 Appointment → 1 Visit_Record
Prescriptions	Refill requests linking a patient to a doctor	FK to Patients, FK to Doctors
Visit_Records	Clinical notes tied to a completed appointment	FK to Appointments
Admins	Administrator accounts with elevated permissions	Manages Doctors, Appointments

3.7 Sample SQL Queries

The following queries represent the core transactions and reports supported by the MediSync backend API.

Create a new patient account

```
INSERT INTO patients (first_name, last_name, email, password_hash, phone,
date_of_birth)
VALUES ('Jordan', 'Alvarez', 'jordan.alvarez@email.com', $1, '555-0142',
'1990-04-12')
RETURNING patient_id;
```

Search available doctors by specialty

```
SELECT d.doctor_id, d.first_name, d.last_name, s.specialty_name, d.location
FROM doctors d
JOIN specialties s ON d.specialty_id = s.specialty_id
WHERE s.specialty_name = 'Cardiology'
ORDER BY d.last_name;
```

Book an appointment (with conflict check)

```
INSERT INTO appointments (patient_id, doctor_id, appt_date, appt_time, status,
reason)
SELECT $1, $2, $3, $4, 'Confirmed', $5
WHERE NOT EXISTS (
  SELECT 1 FROM appointments
  WHERE doctor_id = $2 AND appt_date = $3 AND appt_time = $4
  AND status <> 'Cancelled'
)
RETURNING appointment_id;
```

Cancel an appointment

```
UPDATE appointments
SET status = 'Cancelled', updated_at = NOW()
WHERE appointment_id = $1 AND patient_id = $2;
```

Daily Appointment Report query

```
SELECT a.appt_time, p.first_name || ' ' || p.last_name AS patient,
       d.first_name || ' ' || d.last_name AS doctor, s.specialty_name, a.status
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
JOIN specialties s ON d.specialty_id = s.specialty_id
WHERE a.appt_date = CURRENT_DATE
ORDER BY a.appt_time;
```

Doctor Workload Report query

```
SELECT d.first_name || ' ' || d.last_name AS doctor, s.specialty_name,
       COUNT(a.appointment_id) AS appointments_this_week
```

```
FROM doctors d
JOIN specialties s ON d.specialty_id = s.specialty_id
LEFT JOIN appointments a
  ON a.doctor_id = d.doctor_id
  AND a.appt_date BETWEEN date_trunc('week', CURRENT_DATE) AND CURRENT_DATE + 6
  AND a.status <> 'Cancelled'
GROUP BY d.doctor_id, s.specialty_name
ORDER BY appointments_this_week DESC;
```

Phase 4: Build Your System / Prototype

4.1 Demo Summary

The MediSync prototype is deployed and publicly available at <https://maps-system-three.vercel.app/login>. The live demonstration walks through the full patient-to-administrator workflow described below, showing the front-end interface, the backend database, and real transactions against it.

4.2 Features Demonstrated

Navigation

- Role-based login (Patient, Doctor, Administrator) with a clean, responsive interface built in React, TypeScript, and Tailwind CSS.
- Each role lands on a purpose-built dashboard with clearly labeled navigation matching the menu hierarchy in Phase 2.

User Input (New Transaction)

- Patient registration — creating a new patient profile with contact and insurance information.
- Appointment booking — a patient searches for a doctor by specialty, selects an open time slot, and submits a new appointment, which is written to the Appointments table in real time.

Edit / Change Transaction

- Appointment modification — a patient reschedules or cancels an existing appointment, updating its status in the database.
- Administrator schedule management — an administrator edits a doctor's profile or availability, which is immediately reflected in patient-facing search results.

Query the Data (Reports)

- Administrator dashboard — running the Daily Appointment Report and Doctor Workload Report against the live database, demonstrating the reporting queries documented in Phase 3.
- Appointment history — a patient viewing their own past and upcoming appointments, pulled directly from the Appointments table.

4.3 Demo Story

The demonstration follows a single, connected narrative: a patient logs in and books a new appointment with a cardiologist; the patient then edits that appointment to a different time slot; an administrator logs in, reviews the clinic's appointment volume and doctor workload for the week, and approves a pending appointment. This story is designed to show, in sequence, every requirement outlined in the course guidelines: front-end usability, a new data transaction, an edit to existing data, and a query/report against the backend database — all in one continuous walkthrough.

4.4 Implemented Features (Prototype Scope)

Feature	Status
Responsive login & role-based authentication	Implemented
Patient registration	Implemented
Doctor search by specialty	Implemented
Appointment booking with conflict prevention	Implemented
Appointment editing / cancellation	Implemented
Administrator dashboard	Implemented
Operational reporting	Implemented
Backend database integration (PostgreSQL/Neon)	Implemented
Automated email/SMS reminders	Planned (Future Enhancement)
Analytics/trend dashboard	Planned (Future Enhancement)

4.5 Application Screenshots

The screenshots below were captured directly from the live, deployed MediSync prototype (branded "MAPS" in the running application) at <https://maps-system-three.vercel.app/login>, organized by role to mirror the menu hierarchy in Phase 2.

Login

MAPS

Welcome back
Log in to manage your appointments.

Email

Password

Log in

New patient? [Create an account](#)

Demo logins
Patient: `jdoe@example.com` / `patient123`
Doctor: `mreid@maps.health` / `doctor123`
Admin: `admin@maps.health` / `admin123`

[is-system-three.vercel.app](#)

Figure 7. Role-based login screen with demo credentials for Patient, Doctor, and Administrator accounts.

Patient View

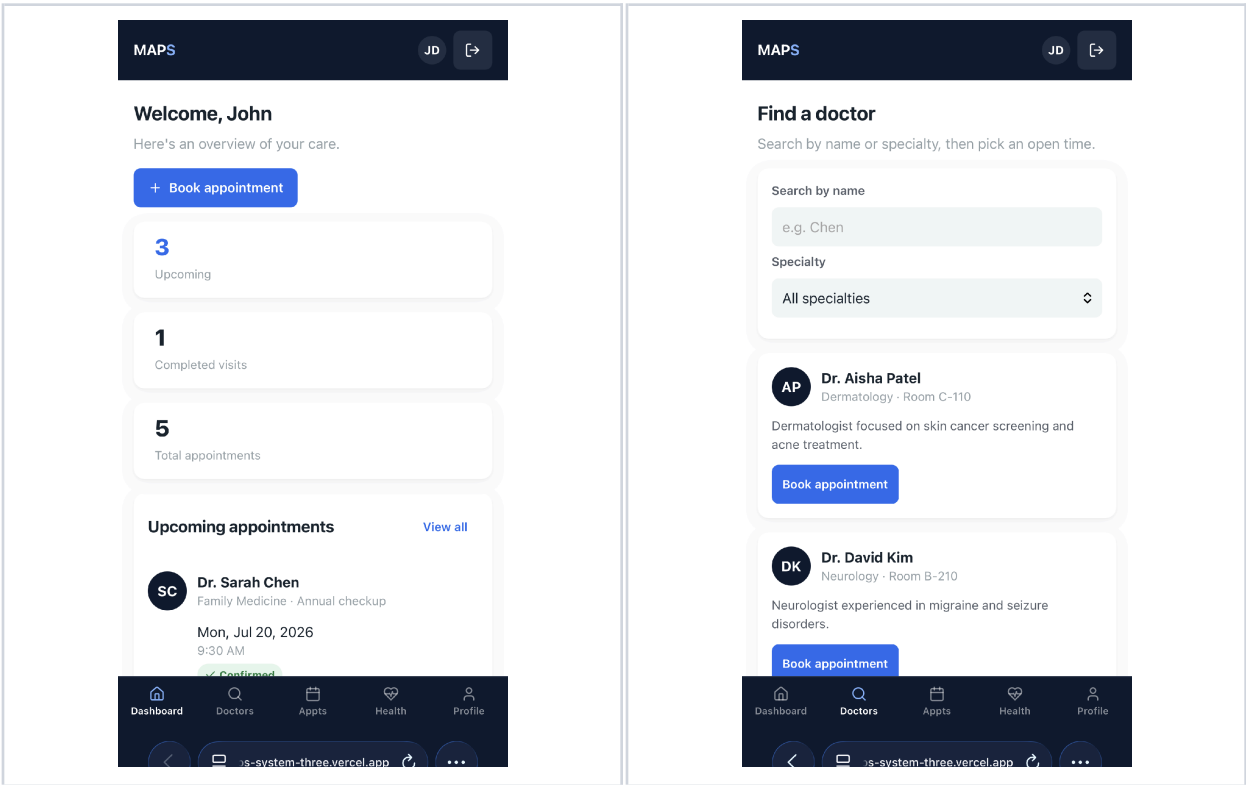


Figure 8. Patient dashboard — upcoming appointments, completed visits, and quick booking access.

Figure 9. Find a Doctor — search by name or specialty, matching the doctor-search use case.

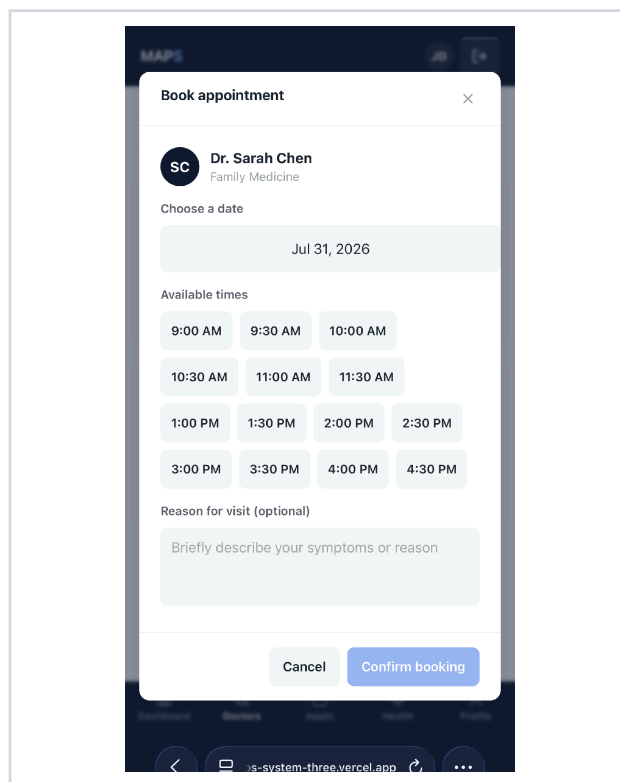


Figure 10. Book Appointment modal — selecting an open time slot for Dr. Sarah Chen.

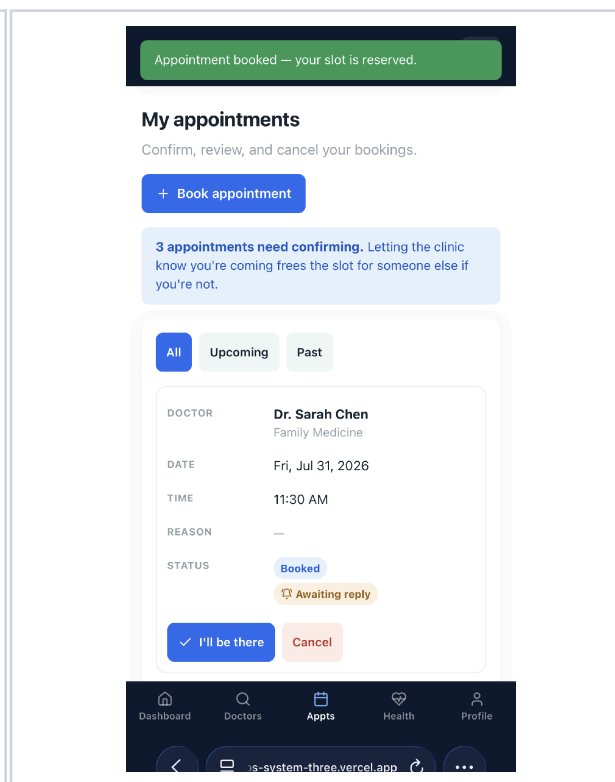


Figure 11. Appointment booked confirmation, with the new booking awaiting patient reply.

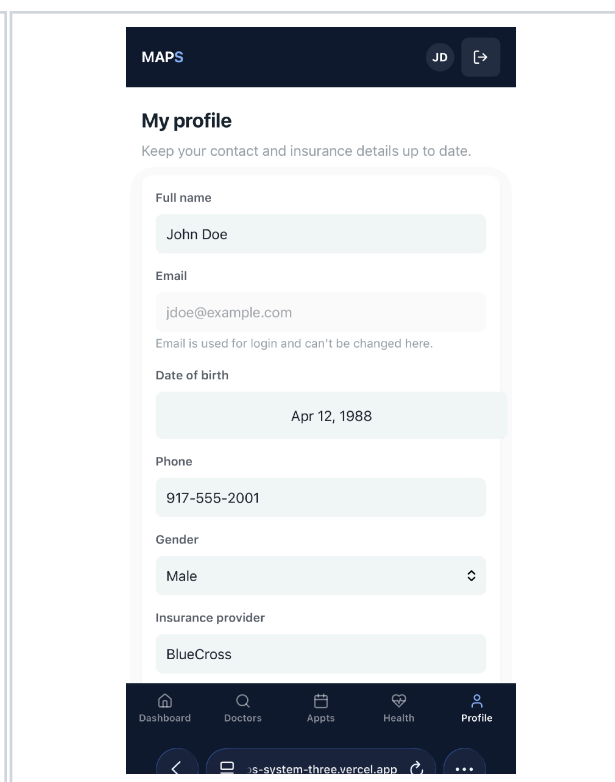
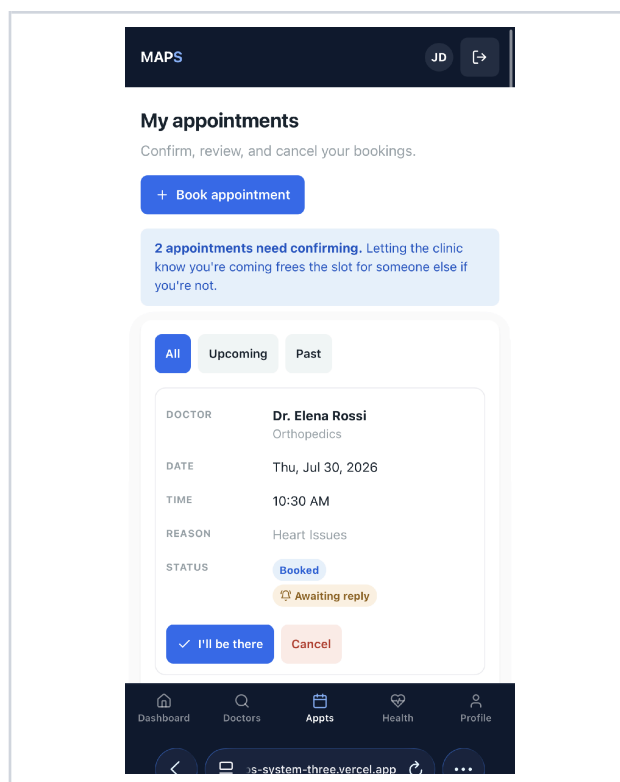


Figure 12. My Appointments — reviewing and confirming a booked visit with Dr. Elena Rossi.

Figure 13. Patient profile — contact, demographic, and insurance information.

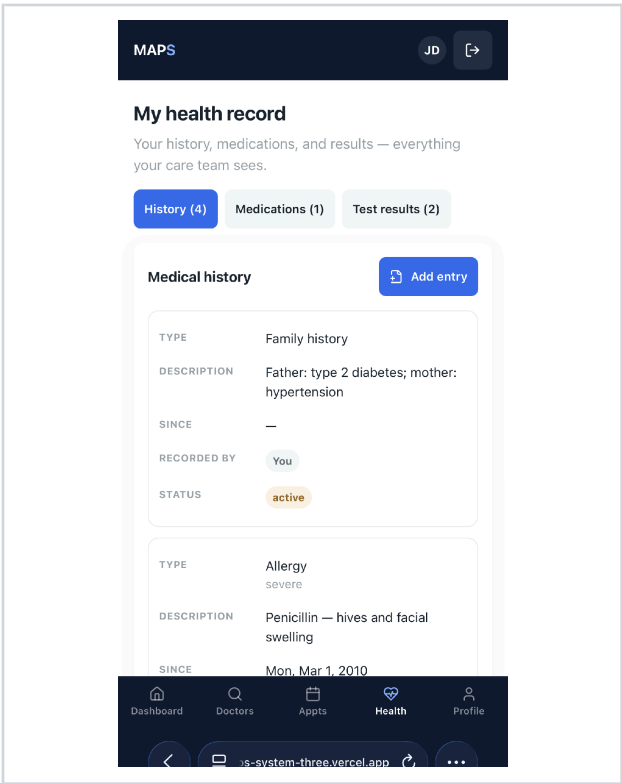


Figure 14. Patient health record — medical history, medications, and test results.

Doctor View

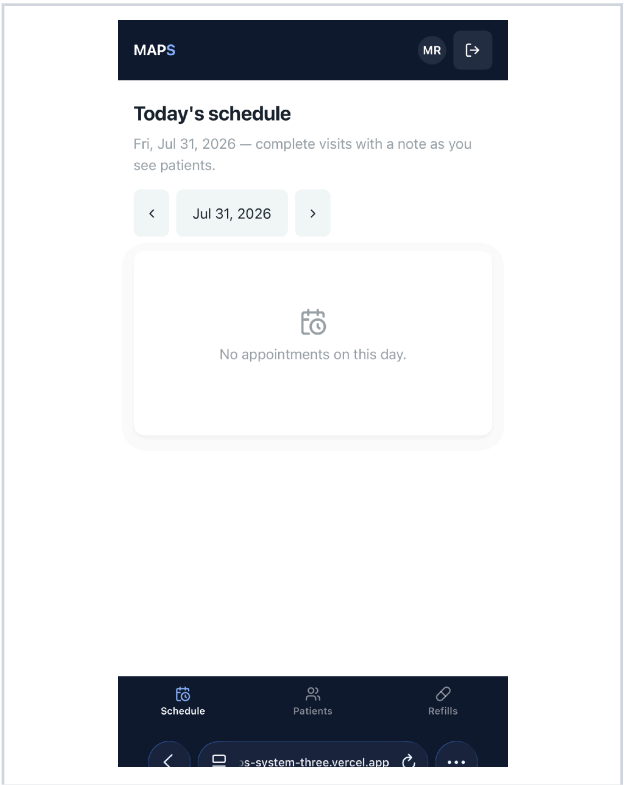


Figure 15. Doctor's daily schedule view for reviewing upcoming visits.

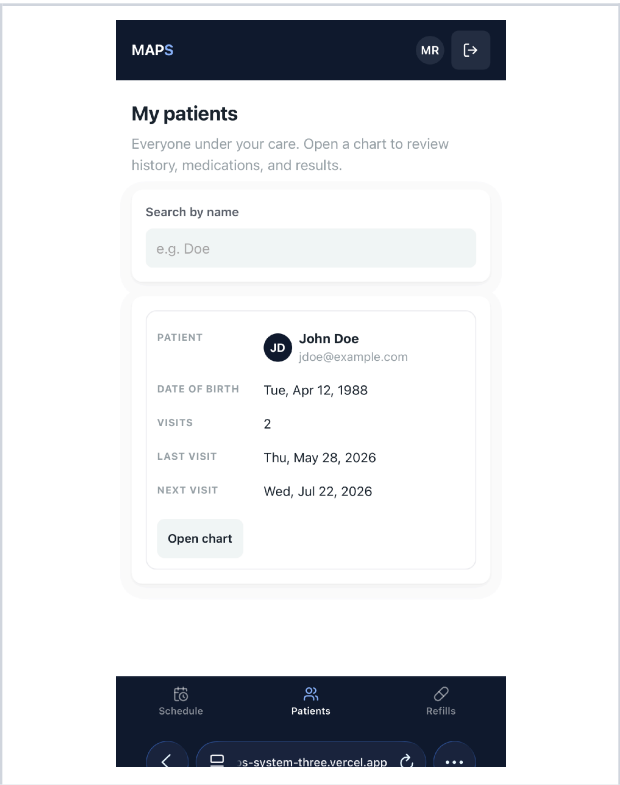


Figure 16. My Patients — doctor's patient roster with visit history and chart access.

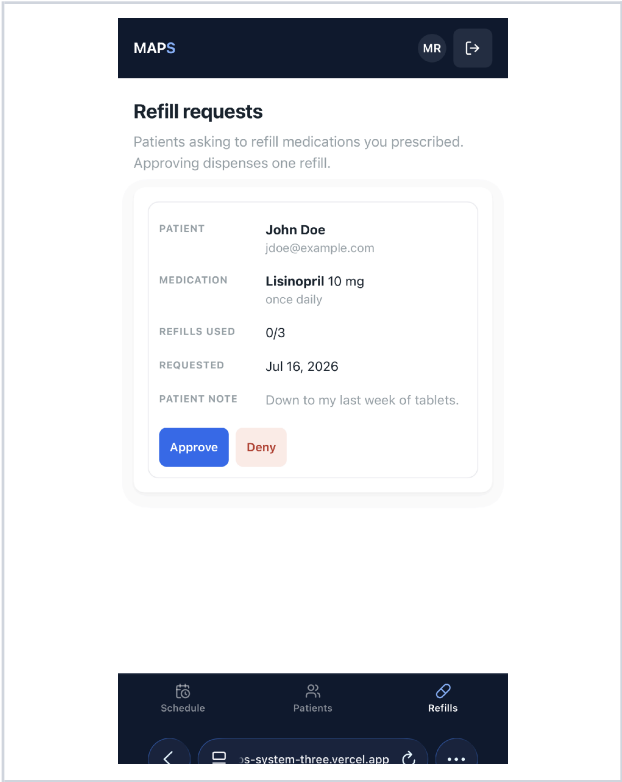


Figure 17. Refill Requests — a doctor approving or denying a patient's prescription refill.

Administrator View

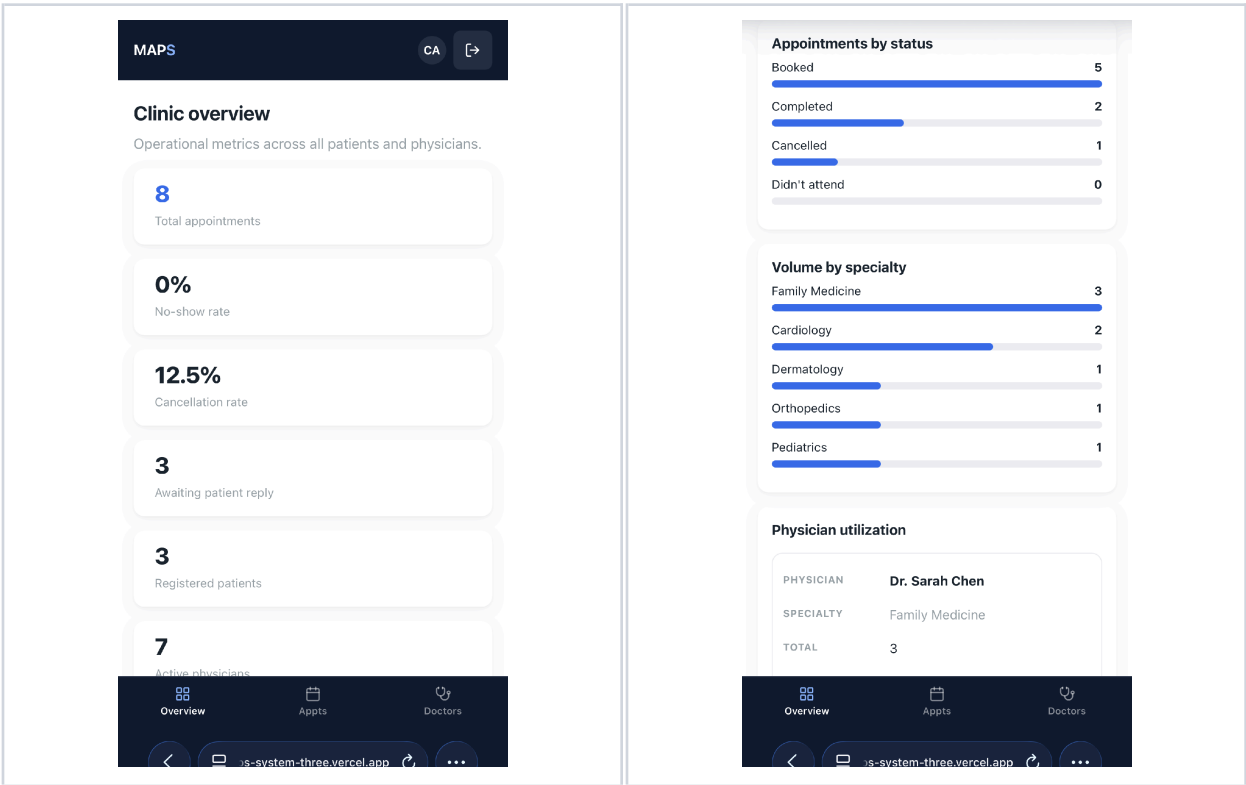


Figure 18. Clinic Overview — total appointments, no-show rate, and cancellation rate at a glance.

Figure 19. Reporting view — appointments by status, volume by specialty, and physician utilization.

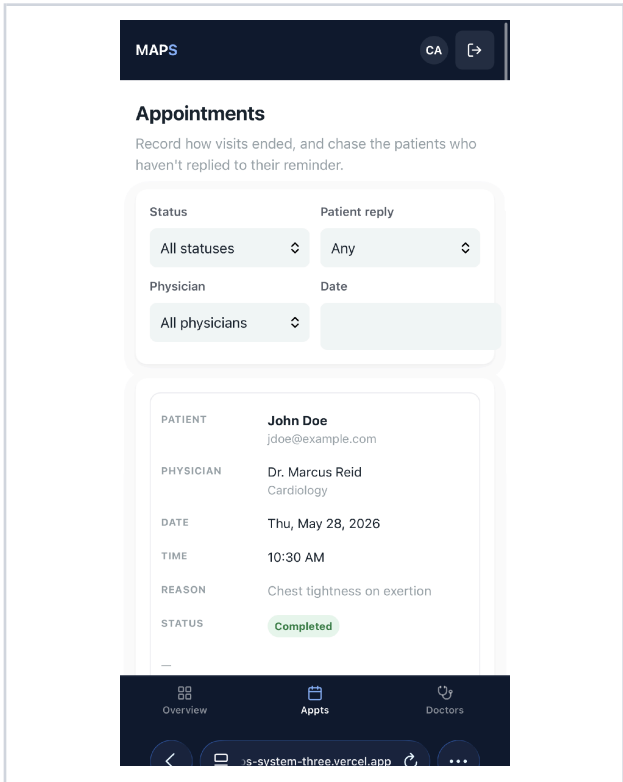


Figure 20. Appointments management — reviewing visit status and details for a specific booking.

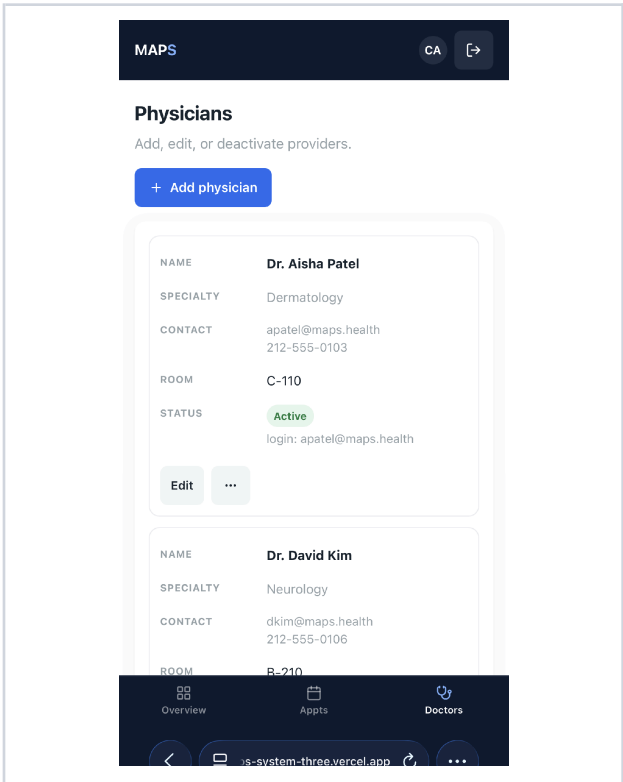
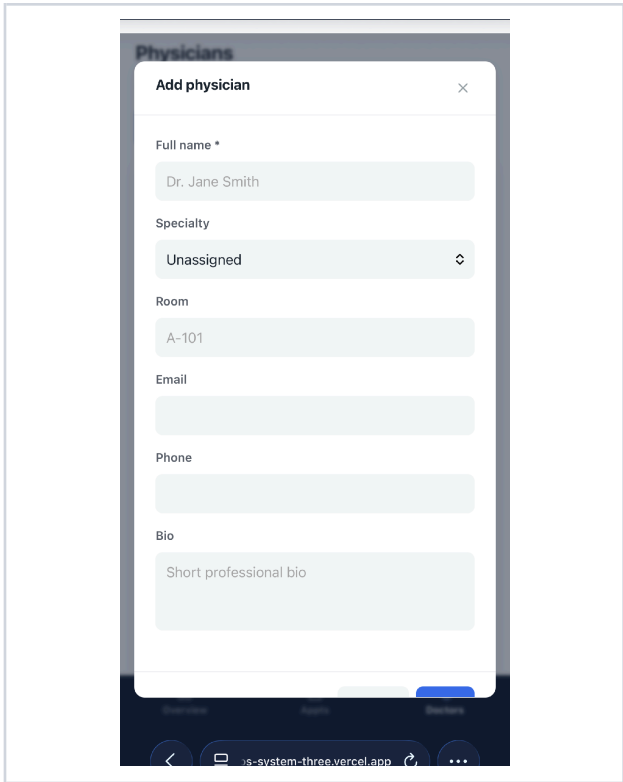


Figure 21. Physicians directory — active provider list with specialty, contact, and room assignment.



*Figure 22. Add Physician modal — administrator onboarding
a new provider into the system.*

Phase 5: System Implementation

5.1 Executive Summary — Lessons Learned

Building MediSync end to end reinforced how tightly systems analysis, database design, UI/UX, and project management are connected in practice. Early time spent on the context diagrams and ERD in Phase 2 paid off directly during development: because the team had already mapped out the data flows and table relationships before writing code, the database schema and API design in Phase 3 required very little rework. The team also learned that scope discipline matters — per the course guidance to "not boil the ocean," the team intentionally deferred features like SMS reminders and an analytics dashboard to keep the 12-week timeline realistic and to make sure the core scheduling workflow was fully functional and demo-ready.

Working as a four-person team surfaced the usual highs and lows of a group project: coordinating around each teammate's coursework schedule required more communication overhead than expected, and merging front-end and back-end work in parallel occasionally produced integration conflicts that had to be resolved through more frequent GitHub check-ins. What worked well was dividing the team along natural front-end/back-end lines while keeping database schema decisions as a shared, whole-team conversation — that structure kept individual workstreams moving while avoiding conflicting assumptions about the data model.

5.2 Hardware / Software

Platform Decisions

The team selected a JavaScript/TypeScript full-stack approach (React + Node.js/Express) over alternatives like C#/.NET or PHP because it allowed the whole team to work in a single language across front end and back end, reducing context-switching and making it easier to review each other's code. PostgreSQL was chosen over a NoSQL option because the data (patients, doctors, appointments) is inherently relational, with clear one-to-many relationships that benefit from foreign-key constraints and referential integrity.

Layer	Technology
Frontend	React, TypeScript, Tailwind CSS
Backend	Node.js, Express (REST API)
Authentication	JWT (JSON Web Tokens) + bcrypt password hashing
Database	PostgreSQL, hosted on Neon
Version Control	Git / GitHub
Deployment / Hosting	Vercel
Development Hardware	Team-owned laptops; no on-premise servers required

5.3 Testing / Training

Test Plan

The team performed functional testing across each major workflow prior to the demo, focused on the scenarios most likely to break in a live demonstration:

- Authentication — verifying that each role (Patient, Doctor, Administrator) can register/login and is routed to the correct dashboard, and that invalid credentials are rejected.
- Appointment creation — confirming that booking a slot writes correctly to the Appointments table and that a second, conflicting booking request is correctly rejected.
- Appointment editing/cancellation — confirming that status changes persist and are reflected immediately in both the patient and administrator views.
- Role-based access control — confirming that a patient cannot access administrator-only routes/screens, and vice versa.
- Database transactions — spot-checking that reporting queries return accurate, up-to-date counts against known test data.

Training Plan

For a production rollout, administrators would be trained first (as the group responsible for onboarding doctors and managing the clinic's day-to-day configuration), followed by doctors, followed by a simple onboarding flow embedded directly in the patient registration process. The most important training focus for administrators is report generation and schedule management; for doctors, it is reviewing the daily schedule and recording visit notes; for patients, the self-service booking flow is designed to be intuitive enough to require no formal training.

5.4 Implementation Plan

MediSync is deployed through Vercel with a PostgreSQL backend hosted on Neon, which allows for a fast, low-friction rollout: there is no on-premise hardware to install, and updates can be deployed continuously through the team's GitHub repository. For a live clinic rollout, the recommended approach is a phased go-live — administrators and doctors onboard first during a one-to-two week configuration period (entering doctor profiles, specialties, and schedules), followed by opening patient self-registration.

5.5 Post-Launch / Support Plan

In production, clinic administrators would serve as the first line of support for day-to-day issues (password resets, schedule corrections), escalating technical issues to the development team through a simple issue-tracking process. Software updates and bug fixes would be delivered through the existing GitHub/Vercel deployment pipeline, allowing fixes to be pushed without downtime.

5.6 Future Modifications / Features

- Automated SMS and email appointment reminders to reduce no-show rates.

- Telemedicine / video-visit integration.
- Insurance eligibility verification at booking time.
- Online co-pay and bill payment.
- A dedicated patient-facing mobile application.
- An expanded analytics dashboard with trend charts for no-show rates, provider utilization, and patient volume over time.

Appendix

This appendix contains the supporting project-management artifacts referenced throughout the report: the project Gantt chart, project charter, meeting logs, change log, risk/issues log, and variance report. Diagrams and reports already presented in the body of the report (context diagrams, use case diagram, ERD, and sample reports) are not repeated here.

Appendix A — Gantt Chart

The chart below shows the full 12-week task-level timeline for the MediSync prototype, grouped into the three development phases described in Phase 3.

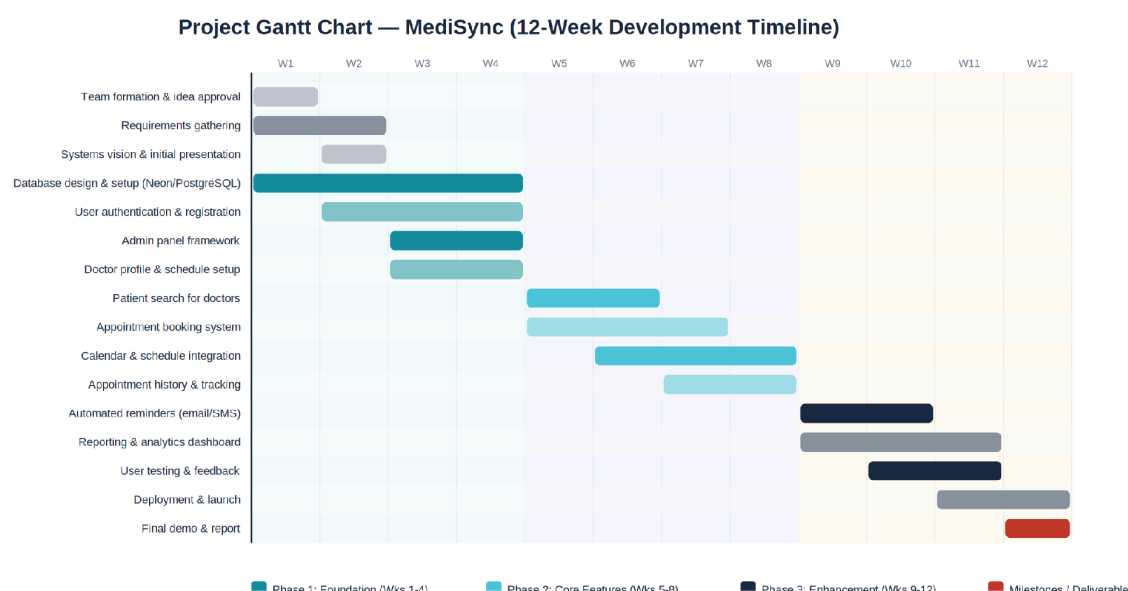


Figure 6. Project Gantt Chart — 12-Week Development Timeline

Appendix B — Project Charter

Item	Detail
Project Name	MediSync — Medical Appointment & Patient Scheduling System
Sponsor / Course	CIS 9590 — Systems Analysis & Design
Team	Gil, Maggie, Mehdi, Raylene (Group 3)
Project Purpose	Replace manual, phone-based clinic scheduling with a centralized, self-service web platform for patients, doctors, and administrators
Objectives	Reduce scheduling conflicts; improve patient access; provide operational reporting for clinic managers
Scope (In)	Patient registration, doctor search, appointment booking/editing, admin schedule management, operational reporting

Item	Detail
Scope (Out)	SMS/email reminders, telemedicine, live payments, mobile app (deferred to future enhancements)
Success Criteria	Working end-to-end demo covering navigation, new transaction, data edit, and report/query, per course requirements
Timeline	12-week prototype development, aligned to course midterm and final deadlines
Authority	Team operates by consensus; database schema decisions made jointly by all four members

Appendix C — Meeting Logs

Date	Attendees	Topics Discussed	Decisions / Action Items
Week 1	All 4 members	Idea brainstorming; reviewed course examples	Selected medical scheduling concept; assigned idea write-up
Week 2	All 4 members	Systems vision document; scope discussion	Finalized patient/doctor/admin function lists
Week 4	All 4 members	Database schema review	Approved 7-table schema (Patients, Doctors, Specialties, Appointments, Prescriptions, Admins, Visit_Records)
Week 6	All 4 members	Front-end/back-end integration check-in	Resolved API contract mismatches; agreed on shared naming conventions
Week 9	All 4 members	Reporting requirements	Finalized the 5 report types and their data sources
Week 11	All 4 members	Testing & demo rehearsal	Walked through demo story end-to-end; logged bugs for fixing
Week 12	All 4 members	Final report assembly & demo prep	Finalized report content and confirmed live deployment

Appendix D — Change Log

Date	Change	Reason
Week 3	Added Prescriptions as a standalone table rather than a field on Appointments	Refill requests needed their own status/lifecycle independent of a specific visit
Week 5	Split "Manage Appointments" into 4 sub-processes in the Level 2 diagram	Original single-process view did not capture conflict-checking logic clearly enough for development

Date	Change	Reason
Week 8	Deferred SMS/email reminders to Phase 3 enhancement backlog	Kept core booking workflow on schedule; reminders required a third-party integration decision
Week 10	Added Provider Utilization Report	Instructor feedback on the first presentation suggested a utilization-focused report for administrators

Appendix E — Issues / Risk Log

Risk / Issue	Likelihood	Impact	Mitigation
Scope creep beyond prototype requirements	Medium	High	Held weekly scope check-ins against the course guidelines; deferred non-core features
Front-end/back-end integration delays	Medium	Medium	Defined API contracts early; frequent GitHub commits and pull-request reviews
Double-booking logic edge cases	Medium	High	Added database-level conflict checks in addition to front-end validation
Team scheduling conflicts (coursework overlap)	High	Low	Used asynchronous updates in shared docs between in-person meetings
Deployment/hosting misconfiguration before demo	Low	High	Deployed early (Week 11) to Vercel to allow time to catch issues before the final demo

Appendix F — Variance Report

Overall, the delivered prototype closely tracked the original scope defined in the Systems Vision Document, with two notable, deliberate variances:

- Reminders and analytics dashboard (originally listed as "future" in the initial presentation) were confirmed as out-of-scope for the prototype and remain future enhancements — this was a planned variance, not a missed deadline.
- The Provider Utilization Report was added after the initial presentation based on instructor feedback, expanding the reporting scope slightly beyond the original four-report plan without affecting the overall timeline.

No major schedule slippage occurred: all three development phases (Foundation, Core Features, Enhancement) were completed within their originally planned week ranges, and the system was deployed ahead of the final demo date to allow for a rehearsal period.